

AI Code Review Platform

Code Sentinel

Code Sentinel is a multi-agent autonomous code review and security intelligence platform. It analyzes code repositories

SEVERITY 100	CONFIDENCE 100	HEALTH 26
-----------------	-------------------	--------------

Language	Python	Architecture	Microservices
Complexity	High	Sec Posture	poor
Bugs Found	7	Vulns Found	7
Doc Grade	—	Prod Ready	not ready

Generated June 24, 2026 at 07:02 UTC
Powered by Groq Llama 3.3 · LangGraph · ChromaDB RAG

The Code Sentinel codebase exhibits a fair level of code quality, but significant concerns exist in terms of security, bug detection, and documentation. The platform's microservices architecture is scalable and flexible, but its complexity may hinder maintenance and debugging. Overall, the codebase requires substantial improvements to ensure reliability, security, and maintainability. The current state of the codebase poses significant risks, particularly in terms of security vulnerabilities and bug detection. Immediate attention is necessary to address these concerns and ensure the platform's production readiness.

Key Findings

Severity	Category	Finding
CRITICAL	Security	Hardcoded demo credentials and insecure JWT secret key expose sensitive information
HIGH	Bugs	Inconsistent error handling and unvalidated user input may lead to unexpected behavior or crashes
MEDIUM	Documentation	Lack of comprehensive documentation, including README and docstring coverage
MEDIUM	Architecture	High complexity of the microservices architecture may make maintenance and debugging challenging

Project	Code Sentinel
Purpose	Code Sentinel is a multi-agent autonomous code review and security intelligence platform. It analyzes c
Architecture	Microservices
Languages	Python, TypeScript, JavaScript, Markdown, YAML, JSON
Frameworks	FastAPI, Next.js
Complexity	High
Entry Points	backend/app/main.py, frontend/next.config.js
Notes	The platform's microservices architecture allows for scalability and flexibility, with each service respons

Codebase Strengths

- ✓ Scalable and flexible microservices architecture
- ✓ Utilization of containerization and orchestration for easy deployment and management

Code Quality: **fair** · Testing Gaps: The codebase lacks comprehensive test coverage, particularly for error scenarios

Uncaught Exception in handleRunEvaluation ■ frontend/components/RagasPanel.tsx Category: unhandled_exception The handleRunEvaluation function in frontend/components/RagasPanel.tsx does not handle all possible exceptions that may occur during the API fetch. This could lead to unhandled exceptions being thrown. <pre>try { await apiFetch(`/api/evaluation/\${jobId}/run`, getToken(), { method: 'POST', }); } catch (err: unknown) { ... }</pre> ■ Fix: Add a catch-all exception handler to handle any unexpected errors.	MEDIUM
Potential Null Pointer Exception in repo_agent.py ■ backend/app/services/agents/repo_agent.py Category: null_reference In backend/app/services/agents/repo_agent.py, the variable 'state' is not checked for null before accessing its attributes. This could lead to a null pointer exception if 'state' is null. <pre>state.get('errors', []) + [error_msg]</pre> ■ Fix: Add a null check for 'state' before accessing its attributes.	HIGH
Inconsistent Error Handling in bug_agent.py ■ backend/app/services/agents/bug_agent.py Category: logic_error In backend/app/services/agents/bug_agent.py, the error handling is inconsistent. In some places, errors are logged and returned, while in other places, they are only logged. This could lead to inconsistent error handling and make it difficult to debug issues. <pre>except Exception as e: error_msg = f'BugAgent failed: {str(e)}' ...</pre> ■ Fix: Standardize the error handling across the codebase.	MEDIUM
Potential Race Condition in AgentTimeline.tsx ■ frontend/components/AgentTimeline.tsx Category: race_condition In frontend/components/AgentTimeline.tsx, the 'isTerminal' variable is accessed and modified by multiple functions. This could lead to a race condition if the functions are executed concurrently. <pre>if (isTerminal.current) { ... }</pre> ■ Fix: Use a lock or a synchronized block to ensure that the 'isTerminal' variable is accessed and modified in a thread-safe manner.	HIGH

Unvalidated User Input in EvaluationPanel.tsx**HIGH**

■ frontend/components/EvaluationPanel.tsx

Category: other

In frontend/components/EvaluationPanel.tsx, the 'jobId' variable is not validated before being used to fetch data. This could lead to security vulnerabilities if the 'jobId' is tampered with.

```
const data = await getEvaluation(jobId);
```

■ **Fix:** Validate the 'jobId' variable before using it to fetch data.

Inconsistent Type Handling in ragas_evaluator.py**MEDIUM**

■ backend/app/services/evaluation/ragas_evaluator.py

Category: type_error

In backend/app/services/evaluation/ragas_evaluator.py, the types of variables are not consistently handled. This could lead to type-related errors.

```
score: float = float(result.get('relevancy_score', 0.5))
```

■ **Fix:** Standardize the type handling across the codebase.

Missing Error Handling in evaluation_router.py**MEDIUM**

■ backend/app/routers/evaluation_router.py

Category: unhandled_exception

In backend/app/routers/evaluation_router.py, the error handling is missing for the get_evaluation function. This could lead to unhandled exceptions being thrown.

```
result = await db.execute(select(EvaluationResult).where(EvaluationResult.job_id == str(job_id)))
```

■ **Fix:** Add error handling for the get_evaluation function.

Critical	High	Medium	Low
2	2	2	1

Hardcoded Demo Credentials

CRITICAL

■ backend/app/utils/auth.py

A07: Authentication Failures

The demo credentials are hardcoded in the authenticate_user function, which is a security risk.

```
return (username == settings.demo_username and password ==
        settings.demo_password)
```

■ Fix: Use environment variables or a secure storage mechanism for credentials.

Insecure JWT Secret Key

CRITICAL

■ backend/app/utils/auth.py

A02: Cryptographic Failures

The JWT secret key is stored in a settings file, which may be exposed to unauthorized access.

```
settings.secret_key
```

■ Fix: Use a secure storage mechanism for the JWT secret key, such as an environment variable or a secrets manager.

Missing Input Validation

HIGH

■ backend/app/routers/auth.py

A04: Insecure Design

The login function does not validate user input, which may lead to security vulnerabilities.

```
if not authenticate_user(payload.username, payload.password):
```

■ Fix: Implement input validation and sanitization for user input.

Insecure Password Storage

HIGH

■ backend/app/utils/auth.py

A02: Cryptographic Failures

The demo password is stored in plaintext, which is a security risk.

```
password == settings.demo_password
```

■ Fix: Use a secure password hashing algorithm to store passwords.

Missing Rate Limiting

MEDIUM

■ backend/app/routers/auth.py

A04: Insecure Design

The login function does not implement rate limiting, which may lead to brute-force attacks.

```
if not authenticate_user(payload.username, payload.password):
```

■ **Fix:** Implement rate limiting for login attempts.

Insecure Token Storage

MEDIUM

■ frontend/lib/api.ts

A05: Security Misconfiguration

The token is stored in local storage, which may be vulnerable to XSS attacks.

```
localStorage.setItem('auth_token', token);
```

■ **Fix:** Use a secure storage mechanism for tokens, such as a cookie with the Secure and HttpOnly flags.

Missing Error Handling

LOW

■ backend/app/routers/auth.py

A09: Logging Failures

The login function does not handle errors properly, which may lead to information disclosure.

```
raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail='Incorrect  
username or password')
```

■ **Fix:** Implement proper error handling and logging mechanisms.

■■ **Exposed Secrets / Credentials**

- demo_username
- demo_password
- secret_key

Overall Grade	—
README Score	0/100
Docstring Coverage	—

#	Category	Effort	Impact	Action
1	Security	medium	high	Remove hardcoded demo credentials and implement secure secret management
2	Bugs	high	high	Implement comprehensive input validation and error handling mechanisms
3	Documentation	medium	medium	Develop and maintain comprehensive documentation, including README and docstrings
4	Architecture	high	medium	Conduct a thorough review of the microservices architecture to identify areas for simplification and improvement

Recommended Next Steps

1. Address critical security vulnerabilities, such as hardcoded demo credentials and insecure JWT secret key
2. Implement comprehensive input validation and error handling mechanisms
3. Develop and maintain comprehensive documentation
4. Conduct a thorough review of the microservices architecture
5. Write additional test cases to ensure comprehensive test coverage

This report was generated automatically by the AI Code Review Platform using Groq Llama 3.3, LangGraph multi-agent orchestration, and ChromaDB RAG retrieval. All findings should be reviewed by a qualified engineer before remediation.